UpsamplingBilinear2d#

https://pytorch.org/docs/stable/generated/torch.nn.UpsamplingBilinear2d.html

Description:

Applies a 2D bilinear upsampling to an input signal composed of several input channels. To specify the scale, it takes either the size or the scale_factor as it’s constructor argument. When size is given, it is the output size of the image (h, w). size (int or Tuple[int, int], optional) – output spatial sizes scale_factor (float or Tuple[float, float], optional) – multiplier for spatial size.

Function Signature

```
class torch.nn.UpsamplingBilinear2d(size=None,
scale_factor=None)[source]#

Parameters

Shape:
```

Code Examples

```
>>> input = torch.arange(1, 5, dtype=torch.float32).view(1, 1,
2, 2)
>>> input
tensor([[[[1., 2.],
          [3., 4.]]]])

>>> m = nn.UpsamplingBilinear2d(scale_factor=2)
>>> m(input)
tensor([[[[1.0000, 1.3333, 1.6667, 2.0000],
          [1.6667, 2.0000, 2.3333, 2.6667],
          [2.3333, 2.6667, 3.0000, 3.3333],
          [3.0000, 3.3333, 3.6667, 4.0000]]]])
```

Notes & Warnings

⚠ **WARNING**

Warning This class is deprecated in favor of interpolate(). It is equivalent to nn.functional.interpolate(..., mode='bilinear', align_corners=True).

Detailed Documentation

Documentation

Applies a 2D bilinear upsampling to an input signal composed of several input channels.

To specify the scale, it takes either the size or the scale_factor as it’s constructor argument.

When size is given, it is the output size of the image (h, w).

size (int or Tuple[int, int], optional) – output spatial sizes

scale_factor (float or Tuple[float, float], optional) – multiplier for spatial size.

This class is deprecated in favor of interpolate(). It is equivalent to nn.functional.interpolate(..., mode='bilinear', align_corners=True).

Input: (N,C,Hin,Win)(N, C, H_{in}, W_{in})(N,C,Hin,Win)

Output: (N,C,Hout,Wout)(N, C, H_{out}, W_{out})(N,C,Hout,Wout) where